Day 3 - API Integration Report - [AVION]

1. API Integration Process

Objective: Integrate Sanity CMS with the frontend to fetch and display product and category data.

Steps:

Set Up Sanity Client:

- Installed the Sanity client library (@sanity/client).
- Configured the client with the project ID, dataset, and API version.
- code:

```
import { createClient } from '@sanity/client';


const client = createClient({
  projectId: 'v39xtyke',
  dataset: 'production',
  apiVersion: '2025-01-13',
  useCdn: true,
});
```

Create GROQ Queries:

- Wrote GROQ queries to fetch products and categories from Sanity.
- code:

```
export async function getAllProducts() {
  const query = `*[_type == "product"] {
    _id,
    title,
    slug,
    productImage {
      asset -> {
        url
      }
    },
    price,
    quantity,
    tags,
    description,
    features,
    dimensions {
      height,
```

```
    width,
    depth
  },
  category -> {
    _id,
    name,
    slug
  }
}`;
return await client.fetch(query);
}
```

Fetch Data in Frontend:

- Used React's useEffect to fetch data on component mount.
- code:

```
const [products, setProducts] = useState<Product[]>([]);

useEffect(() => {
  const fetchProducts = async () => {
    const data = await getAllProducts();
```

```
    setProducts(data);
  };
  fetchProducts();
}, []);
```

Render Data:

- Displayed fetched data in the UI using Tailwind CSS for styling.
- code:

```
<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
  {products.map((product) => (
    <div key={product._id} className="bg-white rounded-lg shadow-
md overflow-hidden">
      <img src={product.productImage.asset.url} alt={product.title} />
      <h2>{product.title}</h2>
      <p>{product.description}</p>
    </div>
  ))}
</div>
```

2. Adjustments Made to Schemas

Product Schema:

- Added fields like productImage, price, quantity, tags, description, features, and dimensions.
- Linked products to categories using references.

```
defineField({
  name: 'category',
  title: 'Category',
  type: 'reference',
  to: [{ type: 'category' }],
});
```

Category Schema:

- Created a category schema with name and slug fields.
- code:

```
defineField({
  name: 'name',
  title: 'Name',
  type: 'string',
  validation: (rule) => rule.required(),
```

```
});
```

3. Migration Steps and Tools Used

Migration Steps:

Export Data:

Exported existing data from the old schema using Sanity's export tool.

Update Schema:

Updated the schema in Sanity Studio to include new fields and relationships.

Import Data:

Imported the updated data into the new schema.

Test Queries:

Tested GROQ queries in Sanity's API Playground to ensure data integrity.

Tools Used:

- **Sanity CLI: For dataset export/import and schema updates.**

- **Sanity Studio: For managing content and schema definitions.**

- **Vercel: For deployment and hosting.**

4. Code Snippets

API Integration:

```
export async function getCategoryBySlug(slug: string) {
  const query = `*[_type == "category" && slug.current == $slug][0] {
    _id,
    name,
    slug
  }`;
  const params = { slug };
  return await client.fetch(query, params);
}
```

Frontend Data Fetching:

```
const [categories, setCategories] = useState<Category[]>([]);


useEffect(() => {
  const fetchCategories = async () => {
    const data = await getAllCategories();
    setCategories(data);
  };
  fetchCategories();
}, []);
```

Schema Definition:

```
defineField({
  name: 'productImage',
  title: 'Product Image',
  type: 'image',
  options: {
    hotspot: true,
  },
});
```

5. Challenges and Solutions

Challenge: Data not appearing in the frontend.

Solution: Verified environment variables and ensured the Sanity client was properly configured.

Challenge: Schema migration errors.

Solution: Used Sanity's dataset export/import tools to migrate data without loss.

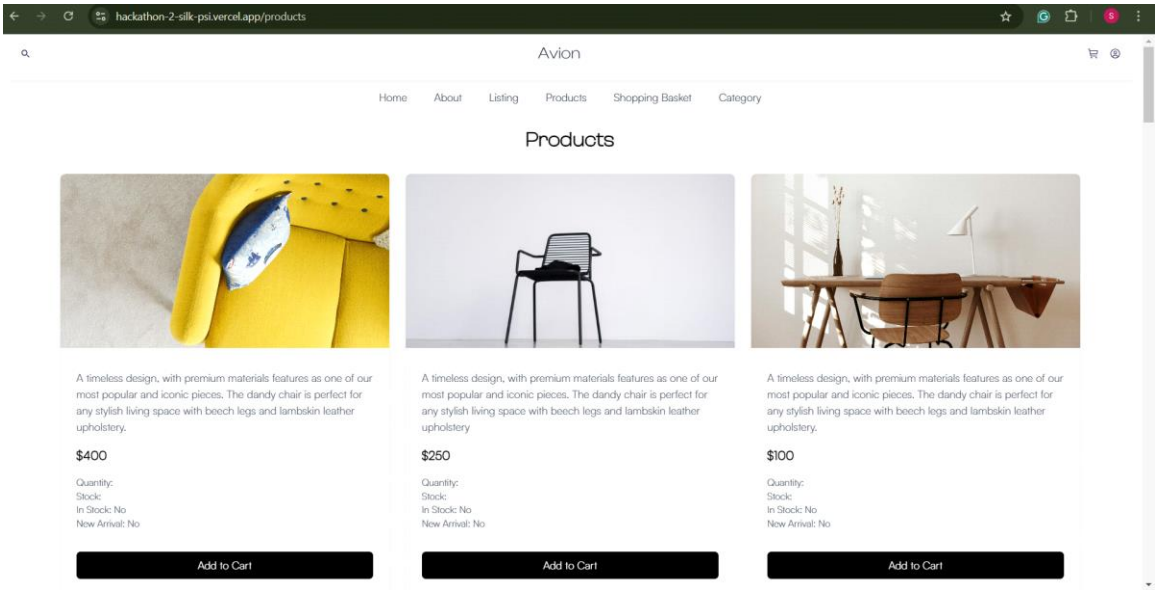
6. Conclusion

- Successfully integrated Sanity CMS with the frontend using GROQ queries and the Sanity client.
- Updated schemas to include necessary fields and relationships.
- Migrated data using Sanity CLI tools and deployed the application using Vercel.

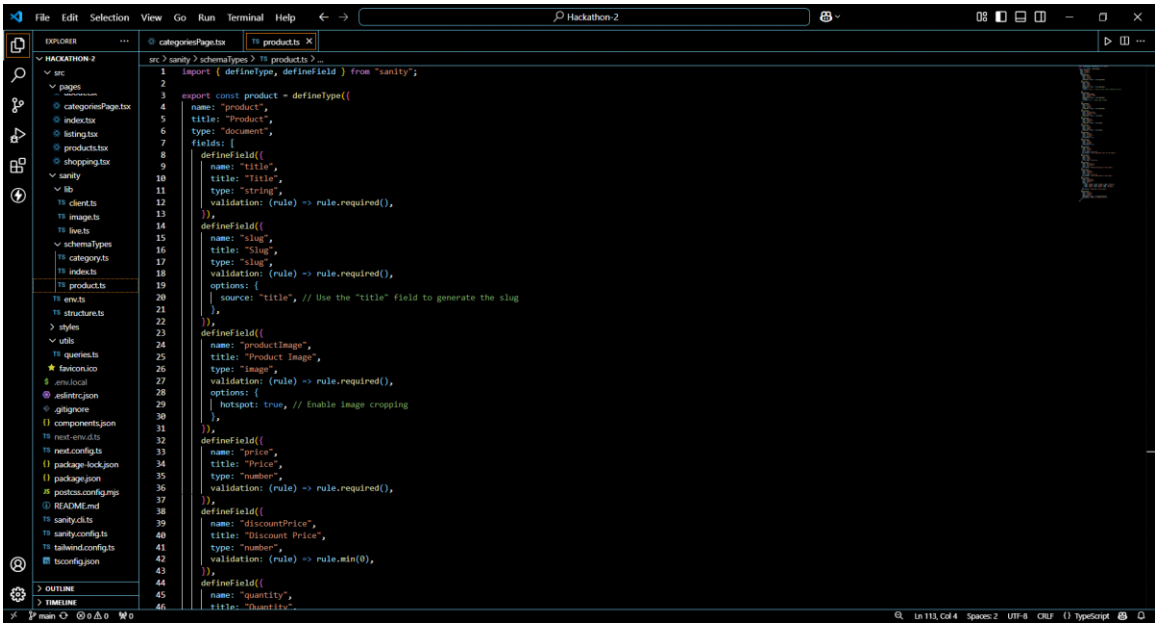
Attachments

Screenshots:

- Frontend Display of Products



- Prodcut's Schema



Code Repository:

- **Link to the GitHub repository: https://github.com/Syeda-Shabia/Hackathon-2**
- **Vercel Link: https://hackathon-2-silk-psi.vercel.app/products**